



KE:

# Inferred Class Hierarchy

Hutchatai Chanlekha

Department of Computer Engineering, Kasetsart University

Email: [fenghtc@ku.ac.th](mailto:fenghtc@ku.ac.th)



# Reasoning for class-subclass testing

- One of the main services offered by a reasoner is to test whether one class is a subclass of another class
- Reasoner can compute the inferred ontology class hierarchy by performing such tests on the classes in an ontology
- In Protégé
  - Manually constructed class hierarchy is called the ***asserted hierarchy***
  - Class hierarchy that is automatically computed by the reasoner is called the ***inferred hierarchy***
- To automatically classify the ontology (and check for inconsistencies)
  - For Protégé 4, invoke via the “Classify...” button in the Reasoner drop down menu
  - For Protégé 5, invoke via “Start reasoner” button in the Reasoner drop down menu

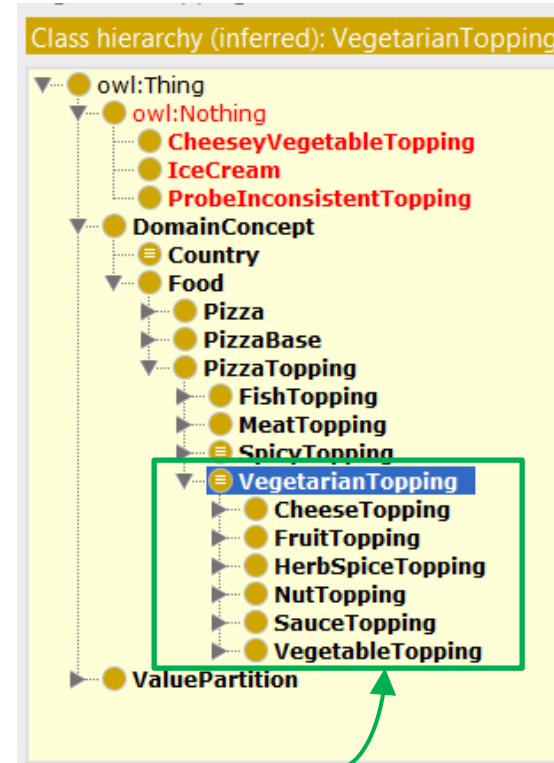


The task of computing the inferred class hierarchy is also known as classifying the ontology.

# Automated Classification [1]

- Automatically compute the class hierarchy using reasoner is one of the major benefits of building an ontology using the OWL-DL
  - When constructing very large ontologies, using a reasoner to compute subclass-superclass relationships becomes almost vital
- In cases where ontologies can have classes that have many superclasses, computing and maintaining multiple inheritance is the job of the reasoner
  - Helps to keep the ontology maintainable
  - Minimize human errors that are inherent in maintaining a multiple inheritance hierarchy

**Try to look by yourself what are other inferred subclasses and why**



Automatically  
inferred subclasses



# Automated Classification [2]

- In general, Reasoner will never place classes as subclasses of primitive classes (i.e. classes without necessary and sufficient conditions)
- Why?



# Create new type of pizza

1. Create MyPizza class as a subclass of NamedPizza
2. Click (+) next to “Subclass of” in the Class description View
3. Choose “Object restriction creator” tab
4. Select property “hasTopping” with restriction type “some” and choose Restriction filter to be “AsparagusTopping”, “MushroomTopping”, “SpinachTopping”, “MozzarellaTopping”

Description: MyPizza

Equivalent To

SubClass Of

- ☒ hasTopping some AsparagusTopping
- ☒ hasTopping some MozzarellaTopping
- ☒ hasTopping some MushroomTopping
- ☒ hasTopping some SpinachTopping
- ☒ NamedPizza



# Do you think MyPizza a Vegetarian Pizza?

## Description: MyPizza

Equivalent To +

SubClass Of +

- hasTopping some AsparagusTopping
- hasTopping some MozzarellaTopping
- hasTopping some MushroomTopping
- hasTopping some SpinachTopping
- NamedPizza

## Description: VegetarianPizzaEquivalent2

Equivalent To +

- Pizza  
and (hasTopping only  
(CheeseTopping or FruitTopping or HerbSpiceTopping or  
NutTopping or SauceTopping or VegetableTopping))

## Description: VegetarianPizza

Equivalent To +

- Pizza  
and (not (hasTopping some FishTopping))  
and (not (hasTopping some MeatTopping))



# Automated Classification and Open World Reasoning

- We believe that MyPizza should be vegetarian pizzas, i.e. subclasses of VegetarianPizza
  - Because they have toppings that are essentially vegetarian toppings

Description: VegetarianPizzaEquivalent2

Equivalent To +

● Pizza  
and (hasTopping only  
(CheeseTopping or FruitTopping or HerbSpiceTopping or  
NutTopping or SauceTopping or VegetableTopping))

Description: VegetarianPizza

Equivalent To +

● Pizza  
and (not (hasTopping some FishTopping))  
and (not (hasTopping some MeatTopping))

Definition of VegetarianPizza

- But MyPizza has not been classified as subclasses of VegetarianPizza
- Why?



# Open World Reasoning

- Reasoning in OWL (Description Logics) is based on what is known as the open world assumption (OWA).
  - Frequently referred to as open world reasoning (OWR).
- With open world assumption, we cannot assume something doesn't exist until it is explicitly stated that it does not exist.
  - Because something hasn't been stated to be true, it cannot be assumed to be false
  - It is assumed that “the knowledge just hasn't been added to the knowledge base”

Because of the open world assumption, until we explicitly say that a MyPizza only has these kinds of toppings, it is assumed (by the reasoner) that a MyPizza could have other toppings.



# What should we do if we want our MyPizza to be classified as Vegetarian Pizza?



## Description: MyPizza

Equivalent To +

SubClass Of +

- hasTopping some AsparagusTopping
- hasTopping some MozzarellaTopping
- hasTopping some MushroomTopping
- hasTopping some SpinachTopping
- NamedPizza

## Description: VegetarianPizzaEquivalent2

Equivalent To +

- Pizza  
and (hasTopping only  
(CheeseTopping or FruitTopping or HerbSpiceTopping or  
NutTopping or SauceTopping or VegetableTopping))

## Description: VegetarianPizza

Equivalent To +

- Pizza  
and (not (hasTopping some FishTopping))  
and (not (hasTopping some MeatTopping))

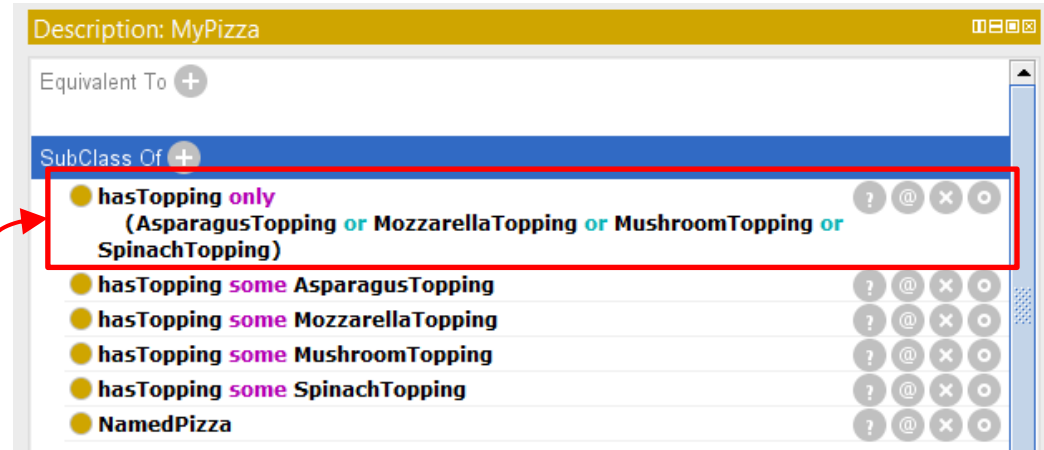


# Closure Axiom

To allow the reasoner to automatically classify MyPizza as subclass of VegetarianPizza, we must add what is known as a closure axiom on the hasTopping property

- In case of our MyPizza

Closure axiom



- A closure axiom on a property consists of a universal restriction that acts along the property to say that it can only be filled by the specified fillers.
  - Normally, the restriction has a filler that is the union of the fillers that occur in the existential restrictions for the property.

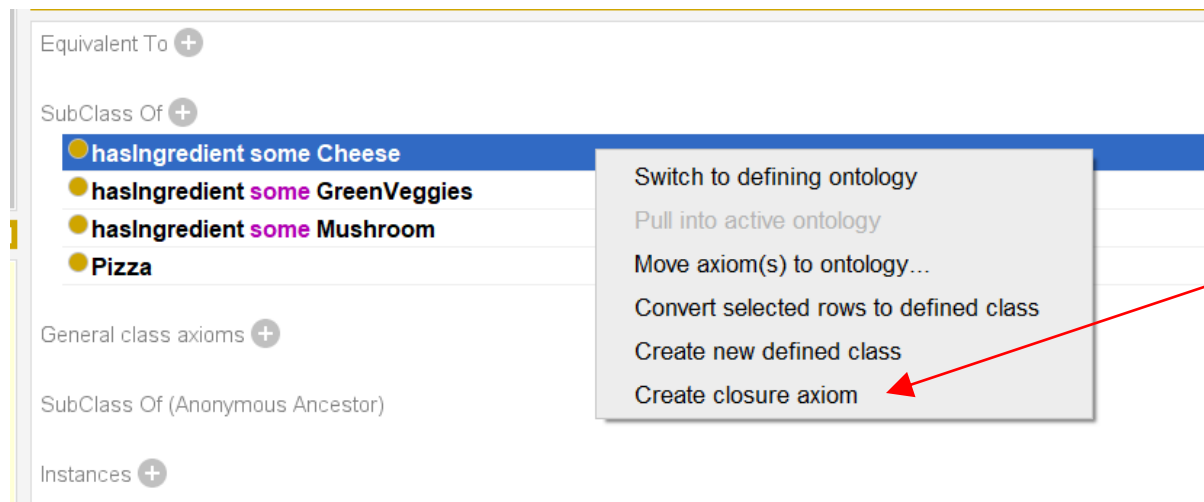


# To create closure axiom

Method 1: Manually add restriction type “only”

Method 2:

- In the “Class description” view, select one of the restrictions
- Right click the restriction and select ‘Create closure axiom’
- You can see that a new closure axiom is created





# What will happen if we add only universal restriction without existential restriction?

- A common error in situations such as above is to only use universal restrictions in descriptions.
- For example, describe MyPizza by making it a subclass of NamedPizza and only using  
hasTopping only (AsparagusTopping or MushroomTopping or MozzarellaTopping or SpinachTopping)

Because of the semantics of the universal restriction, this actually means either: **things that are NamedPizza and only have toppings that are Asparagus, Mushroom, Mozzarella, or Spinach**; OR things that are **NamedPizza and do not have any toppings at all**.